`@Html.Raw()`, which writes the `<script>` tag directly to the document. The user's input has become part of the page's HTML structure. As soon as the page is loaded in a user's browser, the `<script>` tag executes, and the user is compromised. Once an attacker can execute arbitrary JavaScript on a user's browser, they can do pretty much anything.

TIP You can dramatically limit the control an attacker has even if they exploit an XSS vulnerability using a `Content-Security-Policy` (CSP). You can read about CSP at http://mng.bz/nWW2. I have an open-source library you can use to integrate a CSP into your app available on NuGet at http://mng.bz/vnn4.

The vulnerability here is due to rendering the user input in an unsafe way. If the data isn't encoded to make it safe before it's rendered, you could open your users to attack. By default, Razor protects against XSS attacks by HTML-encoding any data written using Tag Helpers, HTML Helpers, or the `@` syntax. So generally you should be safe, as you saw in chapter 17.

Using `@Html.Raw()` is where the danger lies: if the HTML you're rendering contains user input (even indirectly), you could have an XSS vulnerability. By rendering the user input with `@` instead, the content is encoded before it's written to the output, as shown in figure 29.2.